

LLM-BASED DECISION MAKER

Dokumentacja projektowa PZSP2

WERSJA 1

25.10.2025R.

Semestr 2025Z

Zespół nr 18 w składzie:
Kacper Górski
Hanna Biegacz
Jakub Jurczak
Adrian Jędrych

Mentor zespołu: dr. Inż. Krystian Radlak
Właściciel tematu: dr. Inż. Mariusz Kaleta

Spis treści

1 Wprowadzenie	2
1.1 Cel projektu	2
1.2 Wstępna wizja projektu	2
2 Metodologia wytwarzania	3
Podejście Projektowe	3
Analiza ról zespołu	3
3 Analiza wymagań	4
3.1 Wymagania biznesowe	4
3.2 Wymagania użytkownika	5
3.3 Wymagania funkcjonalne	6
3.4 Wymagania niefunkcjonalne	7
3.5 Przypadki użycia	8
3.6 Potwierdzenie zgodności wymagań	8
4 Definicja architektury	9
5 Dane trwałe	9
5.1 Model logiczny danych	9
5.2 Przetwarzanie i przechowywanie danych	9
6 Specyfikacja analityczna i projektowa	9
7 Projekt standardu interfejsu użytkownika	9
8 Specyfikacja testów	9
9 <i>Wirtualizacja/konteneryzacja</i>	10
10 Bezpieczeństwo	10
11 Podręcznik użytkownika	10
12 Podręcznik administratora	10
13 Podsumowanie	10
14 Bibliografia	10

1 Wprowadzenie

1.1 Cel projektu

Projekt zakłada stworzenie inteligentnego systemu, który na podstawie opisu tekstowego oraz dodatkowych dokumentów (RAG) automatycznie generuje model matematyczny i wykonywalny kod w Pythonie lub AMPL. System następnie uruchamia kod, wizualizuje wyniki i umożliwia użytkownikowi interaktywną analizę rozwiązania poprzez zadawanie pytań.

1.2 Wstępna wizja projektu

Projekt ma na celu opracowanie Konwersacyjnego Systemu Wspomagania Decyzji, który implementuje wybrane kroki z metodyki Badań Operacyjnych przy użyciu Dużych Modeli Językowych (LLM'y). Kluczową funkcją systemu jest interaktywna budowa kodu rozwiązującego problem decyzyjny, który użytkownik opisuje w języku naturalnym (korzystając z problemów zawartych w zbiorze testowym NL4OPT, np. „Mam produkcję leków, potrzebuję zoptymalizować...”). System będzie wspierał użytkownika, zadając pytania doprecyzowujące w celu zebrania kompletnych danych i ograniczeń niezbędnych do sformułowania problemu.

Architektura systemu będzie oparta na dekompozycji na wyspecjalizowanych agentów (np. Modeler, Coder, Verifier) , zarządzanych w ramach frameworka agentowego *LangChain*. Proces rozpoczyna się, gdy Analitik wprowadzi zebrany opis problemu, a agent modelujący (LLM) generuje formalny model matematyczny. Użytkownik ma możliwość podglądu, modyfikacji i akceptacji tego modelu. Po akceptacji modelu, system generuje kod wynikowy (np. w *Pythonie* lub *AMPL*), który jest następnie przekazywany do weryfikacji przez użytkownika w roli Programisty.

Po zaakceptowaniu kodu, system umożliwia jego uruchomienie w celu rozwiązania problemu. Wyniki są następnie prezentowane użytkownikowi w roli Decydenta, na przykład w formie raportu zarządczego lub dynamicznej wizualizacji. Wszystkie kroki, wygenerowane modele i odpowiedzi będą zapisywane i wersjonowane w bazie danych. Przewidziana jest również funkcja *RAG* (Retrieval-Augmented Generation), umożliwiająca wgranie plików (np. PDF) z podpowiedziami dla LLM, np. dotyczącymi specyficznej składni lub wzorców modelowania.

2 Metodologia wytwarzania

Organizacja pracy w projekcie odbywa się poprzez repozytorium na Github:

- <https://github.com/Kerciu/conversational-system/>

Jest ono wykorzystywane do przechowywania plików, wspólnej pracy oraz do definiowania i przydzielania zadań za pomocą *Github Issues*. Przydział ten odbywa się dynamicznie, biorąc pod uwagę bieżące możliwości i umiejętności członków zespołu.

Komunikacja wewnętrzna odbywa się przy pomocy Messengera i Discorda. Do kontaktu z mentorem, klientem oraz ekspertami służą Teams i Leon. Dodatkowo wykorzystywana Miro jest przestrzenią roboczą do tworzenia modeli i diagramów.

Podejście Projektowe

Projekt realizowany jest w metodyce zwinnej (*Agile*), w której proces wytwórczy podzielony jest na dwutygodniowe iteracje, zwane sprintami. Na początku każdego cyklu zespół wspólnie definiuje cele, co pozwala na systematyczne dostarczanie kolejnych funkcjonalności i elastyczne reagowanie na pojawiające się wyzwania. Model pracy opiera się na samoorganizacji zespołu i kolektywnym podejmowaniu decyzji, co sprzyja efektywnej współpracy i współdzieleniu odpowiedzialności za realizację zadań.

Analiza ról zespołu

W ramach ćwiczeń nr 2 wykonano test Belbina i wyłoniono po 3 najlepiej pasujące role dla każdego członka. Uzyskane wyniki są następujące:

- Completer / Finisher — x3
- Shaper — x2
- Team Worker — x1
- Coordinator — x1
- Implementer — x1
- Plant — x1
- Resource Investigator — x1
- Evaluator — x1
- Specialist — x1

Powyższe role pokrywają niemal wszystkie dostępne z puli, co przekłada się na wysoką kompatybilność oraz dobrą dystrybucję ról w zespole.

3 Analiza wymagań

3.1 Wymagania biznesowe

Nadrzędnym celem biznesowym jest przyspieszenie i obniżenie bariery wejścia w proces rozwiązywania problemów decyzyjnych poprzez stworzenie Konwersacyjnego Systemu Wspomagania Decyzji. System ten ma automatyzować kluczowe etapy, takie jak formalizacja problemu, modelowanie matematyczne, generowanie kodu oraz wizualizacja rozwiązań. Rozwiązanie ma skrócić czas od opisu problemu biznesowego (np. "potrzebuję zoptymalizować produkcję leków") do uzyskania zweryfikowanego, zoptymalizowanego rozwiązania zadanego problemu i zrozumiałego raportu dla klienta.

3.2 Wymagania użytkownika

Priorytet	Rola	Wymaganie
Wysoki	Klient	Klient ma możliwość wprowadzenia opisu problemu decyzyjnego w języku naturalnym – polski bądź angielski o długości nie większej niż 1000 znaków (np. „Mam taką produkcję i takie zasoby...”).
Średni	Klient	Klient może zweryfikować poprawność i kompletność danych wejściowych oraz komunikaty o brakach lub błędach.
Średni	Klient	Użytkownik ma możliwość przeglądania, edycji lub akceptacji wygenerowanego modelu matematycznego.
Średni	Klient	Użytkownik ma możliwość akceptacji lub edycji wygenerowanego kodu przed jego uruchomieniem.
Średni	Klient	Klient ma możliwość wyboru różnych wersji agentów LLM, o ile dostarczy do nich uprawnienia (podstawowy agent, zapewniony na czas trwania semestru 25Z – Google Gemini).
Wysoki	Klient	Klient może dodać materiały w zatwierdzonym przez obie strony formacie (np. plików PDF / tekstowych) jako dodatkowe dane wejściowe – moduł RAG.
Średni	Klient	Klient może przejść między etapami (modelowanie, kod, wizualizacja) i agentami systemu bez restartu i utraty rozmowy.
Niski	Klient	Użytkownik może wyeksportować wyniki, kod lub raport końcowy w formacie PDF/DOCX.
Wysoki	Klient	Klient może powrócić do wcześniej utworzonej sesji (konwersacji) i wczytać jej stan (wygenerowany, zaakceptowany model lub kod, itp.).

3.3 Wymagania funkcjonalne

1. Zarządzanie Problemem Decyzyjnym

1.1: System musi udostępniać interfejs (np. pole tekstowe) do wprowadzania opisu (max 1000 znaków) problemu decyzyjnego w języku naturalnym (pl/eng) przez klienta.

1.2: System, wykorzystując LLM, musi mieć możliwość analizy wstępnego opisu i zadawania klientowi pytań doprecyzowujących, jeśli wykryje brakujące dane lub niejasności.

1.3: System musi umożliwiać klientowi wgranie (upload) w zatwierdzonym przez obie strony formacie plików (np. PDF/txt), które posłużą jako kontekst (baza wiedzy RAG) dla LLM podczas generowania modelu lub kodu.

2. Modelowanie Matematyczne

2.1: System musi, na żądanie klienta, wysłać przetworzony opis problemu do LLM (agenta modelującego) w celu wygenerowania sformalizowanego modelu matematycznego.

2.2: System musi wyświetlić klientowi wygenerowany model matematyczny.

2.3: System musi pozwalać klientowi na iteracyjne poprawianie modelu poprzez dalsze prompty (np. "Nie podoba mi się ten fragment, zmień go...").

2.4: System musi rejestrować formalną akceptację lub odrzucenie modelu przez klienta. Akceptacja jest warunkiem przejścia do etapu generowania kodu.

3. Generowanie i Weryfikacja Kodu

3.1: Po zaakceptowaniu modelu matematycznego, system musi automatycznie (lub na żądanie) przekazać model do LLM (agenta kodującego) w celu wygenerowania kodu wynikowego (np. w Pythonie z bibliotekami lub w języku AMPL).

3.2: System może wyświetlić wygenerowany kod użytkownikowi.

3.3: System musi pozwolić klientowi na zaakceptowanie lub odrzucenie wygenerowanego kodu.

4. Wykonanie i Prezentacja Wyników

4.1: System musi uruchomić ten kod w celu rozwiązania problemu.

4.2: System musi przechwycić i zapisać wyniki (dane wyjściowe) zwrócone przez wykonany kod.

4.3: System musi prezentować wyniki klientowi w zrozumiałej formie, np. jako tekstowy raport zarządczy lub wizualizacja (np. wykres).

5. Administracja

5.1: System musi zapisywać w bazie danych wszystkie kluczowe artefakty procesu: prompty użytkownika, odpowiedzi LLM, wygenerowane modele matematyczne, wygenerowany kod oraz uzyskane wyniki i wyświetlać historię utworzonych sesji, wyświetlając w. w. elementy oraz ich stan.

5.2: System musi posiadać mechanizm uwierzytelniania użytkowników.

5.3: System musi umożliwiać tworzenie i edycję kont użytkowników.

3.4 Wymagania niefunkcjonalne

1. Architektura i Rozszerzalność

1.1: Architektura systemu musi być modularna i oparta na dekompozycji na wyspecjalizowanych agentów (np. Modeler, Coder, Verifier), zgodnie z kryteriami oceny projektu.

1.2: Architektura musi pozwalać na późniejszy rozwój i łatwe dodawanie nowych agentów (np. Explainer Agent) lub wymianę modeli LLM.

2. Bezpieczeństwo

2.1: Mechanizm uruchamiania kodu wygenerowanego przez LLM musi być w pełni izolowany (sandboxed), aby zapobiec atakom typu Code Injection i nieautoryzowanemu dostępowi do systemu operacyjnego serwera.

2.2: Klucze API do zewnętrznych usług LLM (np. Google Gemini, Groq) muszą być przechowywane w bezpieczny sposób (np. jako zmienne środowiskowe lub w systemie zarządzania sekretami).

2.3: Dane użytkownika użyte do tworzenie konta i późniejszego logowania muszą być przechowywane w sposób zapewniający ich integralność i poufność.

3. Użyteczność i Dostępność

3.1: System musi być aplikacją webową.

3.2: Interfejs użytkownika musi być intuicyjny.

3.3: System musi jasno komunikować użytkownikowi aktualny status procesu (np. "Generowanie modelu...", "Wykonywanie kodu...").

4. Wydajność

4.1: Czas oczekiwania na odpowiedź konwersacyjną (np. pytanie doprecyzowujące od LLM) nie powinien zniechęcać użytkownika do interakcji.

4.2: Czas generowania modelu lub kodu dla problemów o średniej złożoności nie powinien przekraczać zdefiniowanego progu (np. 60 sekund).

4.3: System musi być w stanie obsłużyć upload plików RAG (PDF) o rozmiarze co najmniej 10 MB.

5. Niezawodność i Integralność

5.1: System musi poprawnie obsługiwać błędy zwracane przez API LLM (np. błędy limitu tokenów, niedostępność usługi).

5.2: System musi poprawnie obsługiwać błędy wykonania kodu (np. błędy składniowe, błędy solvera).

6. Ograniczenia Technologiczne

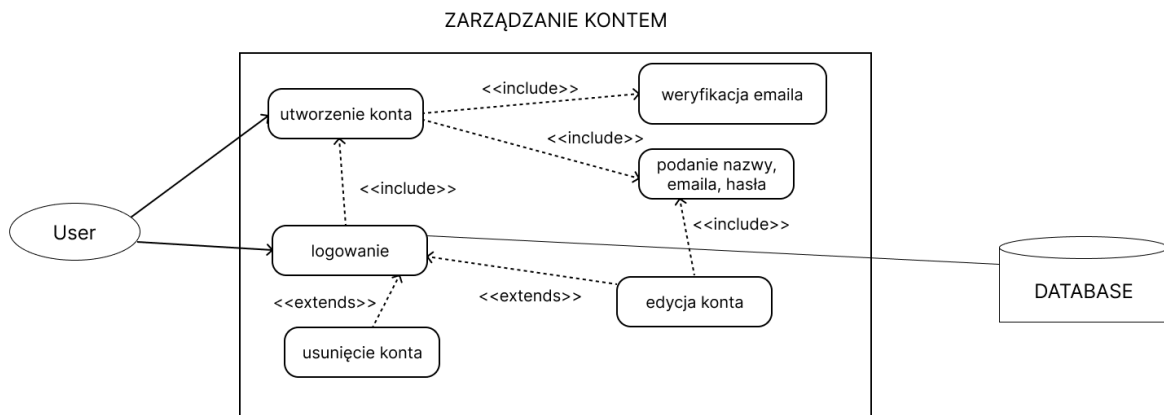
6.1: System musi korzystać z bazy danych do przechowywania konwersacji i modeli.

6.2: System musi być zdolny do integracji z różnymi dostawcami LLM (np. darmowymi jak Google Gemini/Groq lub płatnymi).

3.5 Przypadki użycia

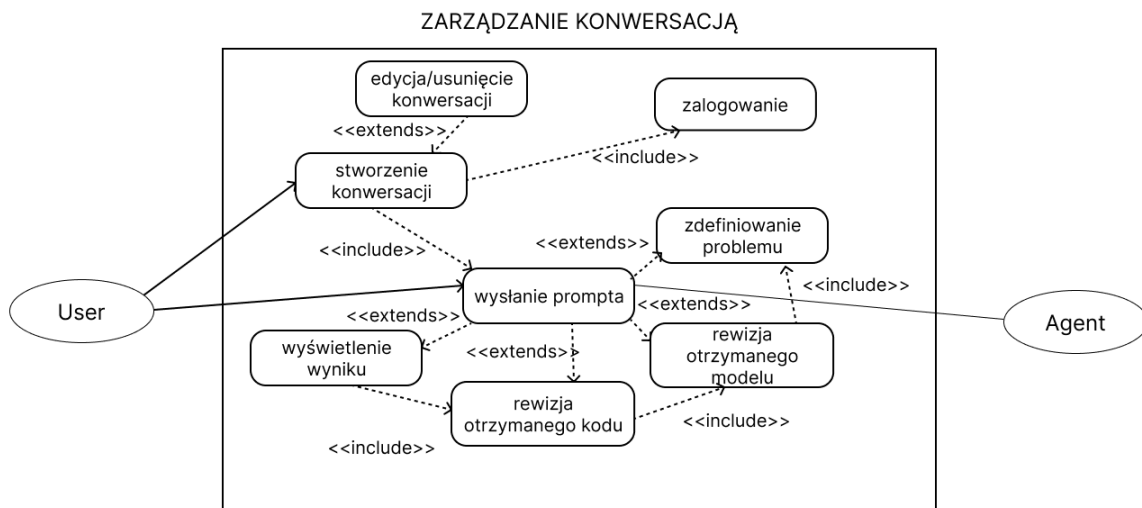
3.5.1

Diagram przypadku zarządzania kontem



3.5.2

Diagram przypadku zarządzania konwersacją



3.6 Potwierdzenie zgodności wymagań

Jurczak Jakub 2 (STUD) poniedziałek 19:25



Zatwierdzenie wymagań

Dzień dobry,

W plikach kanału zamieściliśmy finalną wersję dokumentacji wstępnej. [Kaleta Mariusz](#), [Radlak Krystian](#) czy możemy prosić o przejrzenie ich i w przypadku braku zastrzeżeń zaakceptowanie w formie odpowiedzi na ten post?



Otwórz 4 odpowiedzi od użytkowników [Kaleta Mariusz](#), ty i [Radlak Krystian](#)

Radlak Krystian piątek 09:06

Dzień dobry,

ja nie mam uwag. Prosiłbym Państwa do odniesienia się do uwag [Kaleta Mariusz](#)

Jurczak Jakub 2 (STUD) 13:34

Dzień dobry, dodaliśmy ostatnią funkcjonalność i prosimy o jeszcze jedno spojrzenie.

Kaleta Mariusz 14:34

Tak, potwierdzam, że o to chodziło. Akcept.



Odpowiedz

4 Definicja architektury

[plan struktury systemu – model komponentów, modułów, serwisów

zastosowane szablony architektoniczne (np. MVC, mikroserwisy, SOA, szyna usług)

interfejsy kluczowych elementów struktury oraz wyjaśnienie połączeń i interakcji pomiędzy nimi

perspektywa logiczna/funkcjonalna oraz perspektywa sprzętowa (tzw. deployment): systemy operacyjne, serwer aplikacyjny, system bazy danych i inne mechanizmy utrwalania danych, system raportowania, system analityczny/BI, mechanizmy zarządzania, mechanizmy bezpieczeństwa

diagramy]

Diagram kontekstu:

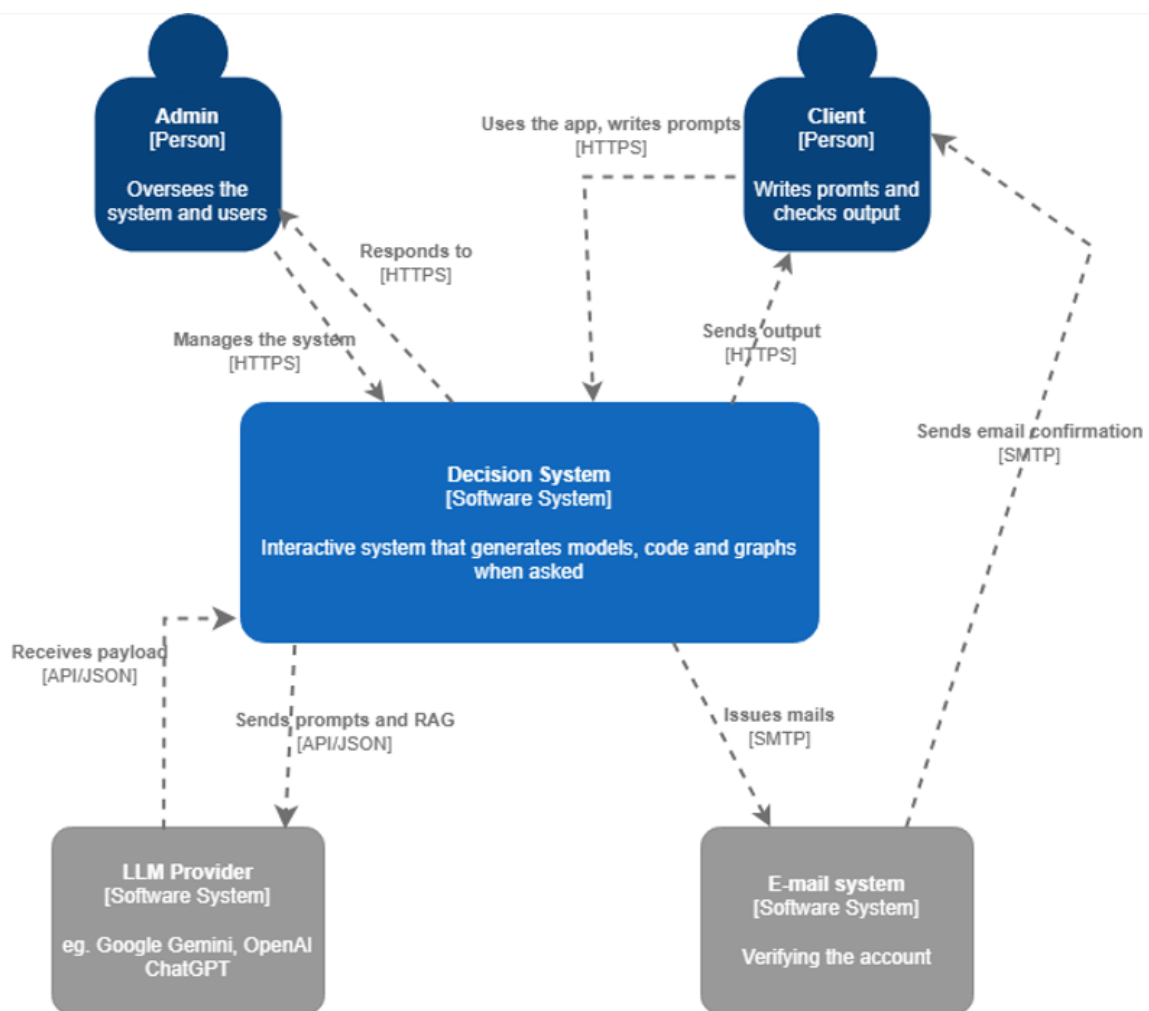


Diagram kontenerów:

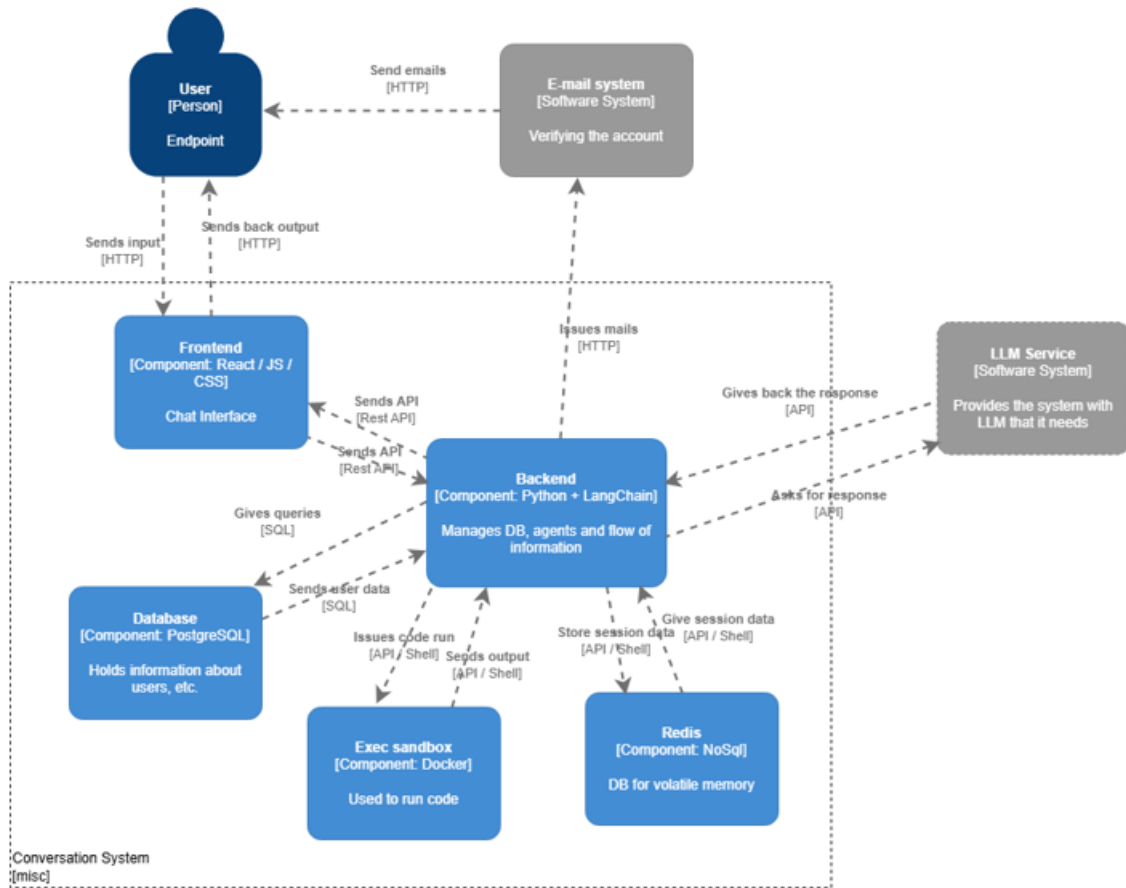
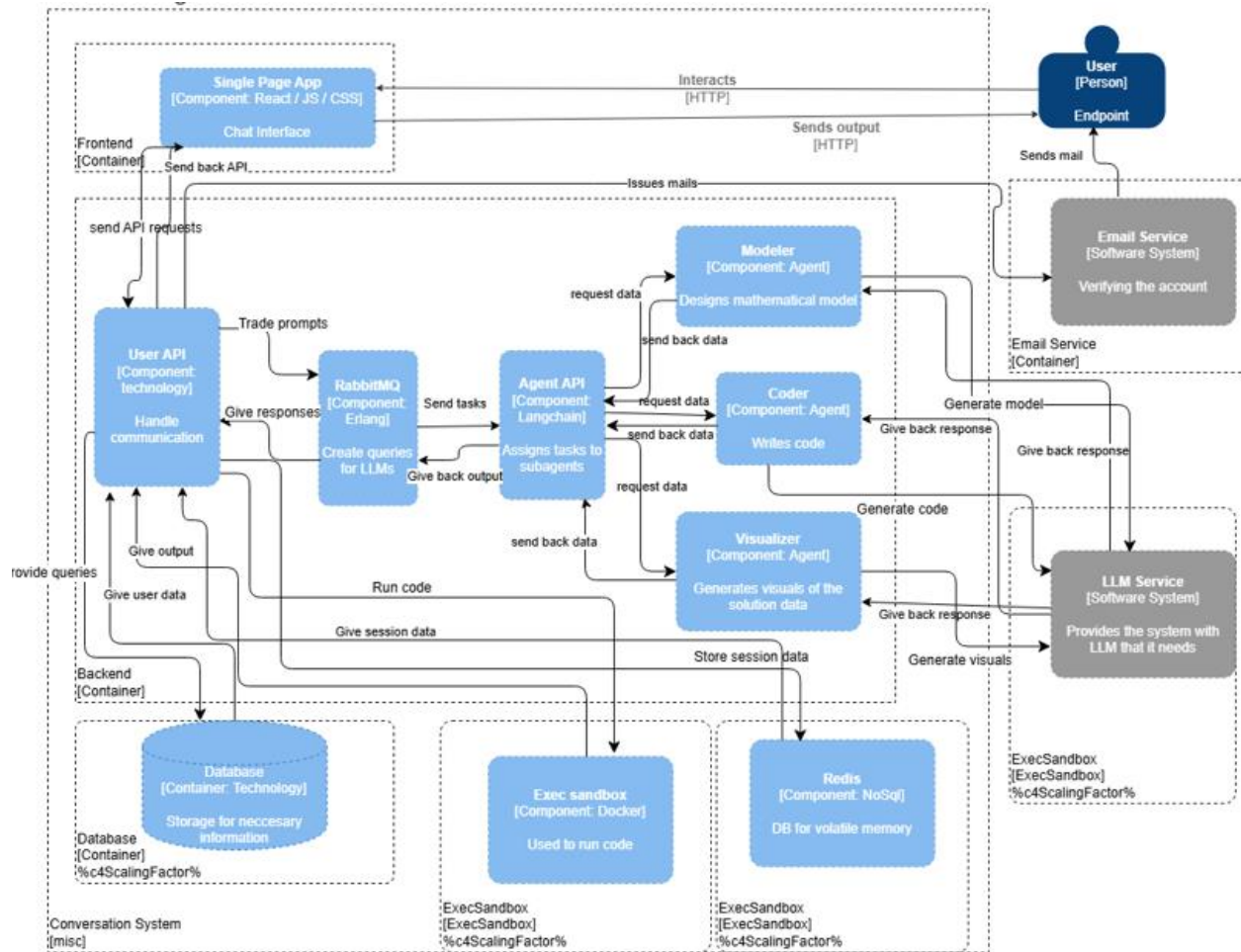


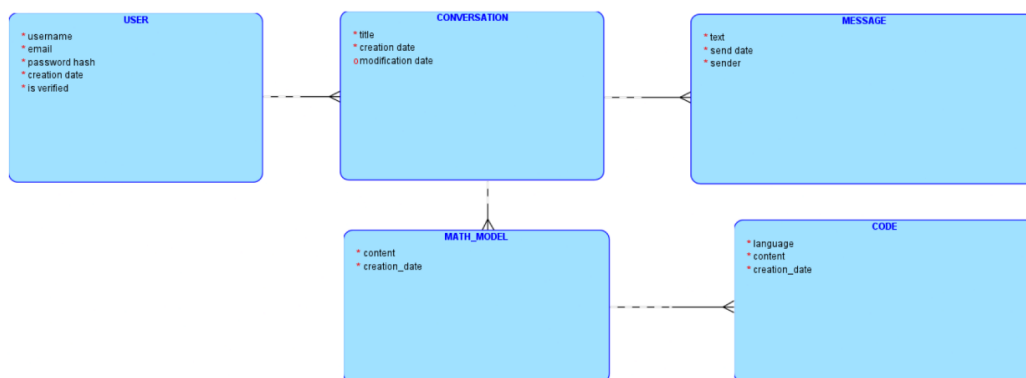
Diagram komponentów:



5 Dane trwał

5.1 Model logiczny danych

Model logiczny



5.2 Przetwarzanie i przechowywanie danych

Zaprojektowana baza danych pełni rolę trwałego magazynu dla Konwersacyjnego Systemu Wspomagania Decyzji, gromadząc informacje o użytkownikach oraz historię ich interakcji z agentami AI. System archiwizuje proces rozwiązywania problemów optymalizacyjnych, przechowując zdefiniowane zadania, modele matematyczne i wygenerowany kod źródłowy w ramach wątków konwersacji.

Dodatkowo, pamięć podręczna Redis obsługuje dane tymczasowe, takie jak kody weryfikacyjne, wymagające

Poniżej znajduje się krótki opis poszczególnych encji znajdujących się w naszej bazie danych:

- **USER** – przechowuje podstawowe dane kont użytkowników systemu (login, e-mail, hasło, status weryfikacji).
- **CONVERSATION** – reprezentuje pojedynczą sesję konwersacyjną użytkownika z systemem, z tytułem i znacznikami czasu utworzenia oraz modyfikacji.
- **MESSAGE** – zapisuje kolejne wiadomości w ramach rozmowy, obejmując treść, datę wysłania oraz informację o nadawcy (użytkownik lub agent AI)
- **MATH_MODEL** – przechowuje zbudowane przez system formalne modele matematyczne powiązane z daną konwersacją.
- **CODE** – zawiera wygenerowany kod źródłowy (np. Python, AMPL) odpowiadający konkretnemu modelowi matematycznemu. Relacje pomiędzy encjami odzwierciedlają przepływ pracy: użytkownik posiada wiele konwersacji, każda konwersacja zawiera wiele wiadomości i powiązane z nią wersje modeli matematycznych, a każdy model może mieć wiele powiązanych artefaktów kodu.

6 Specyfikacja analityczna i projektowa

6.1 Link do repozytorium kodu

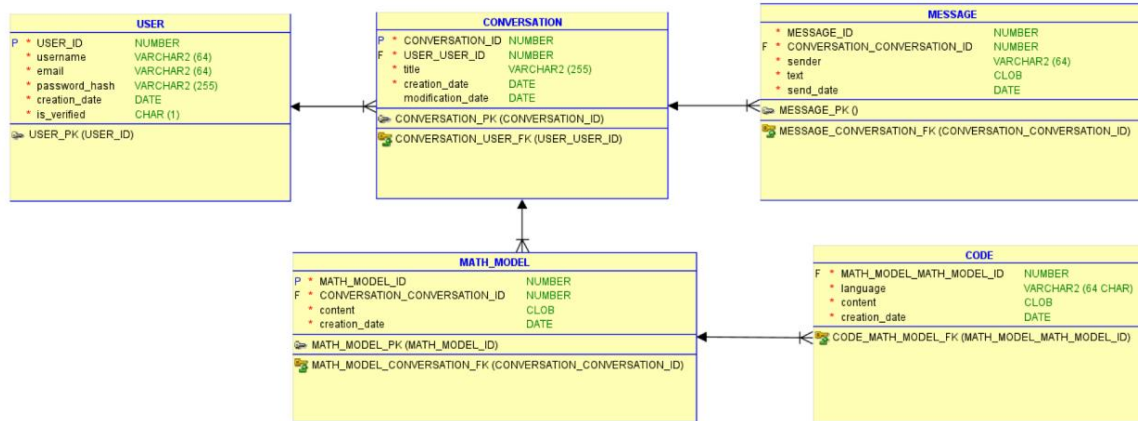
<https://github.com/Kerciu/conversational-system/>

6.2 Metody realizacji:

- **Języki programowania:** Python (agent-service), Java (backend), TypeScript (frontend).
- **Backend Framework:** Spring Boot.
- **Frontend Framework:** React.
- **Baza danych:** PostgreSQL.
- **Cache/Broker:** Redis, RabbitMQ (do komunikacji między serwisami agentów).
- **Konteneryzacja:** Docker, Docker Compose.
- **Integracja LLM:** langchain.

6.3 Model relacyjny:

Model relacyjny



6.4 Statystyki

Liczba plików: 221

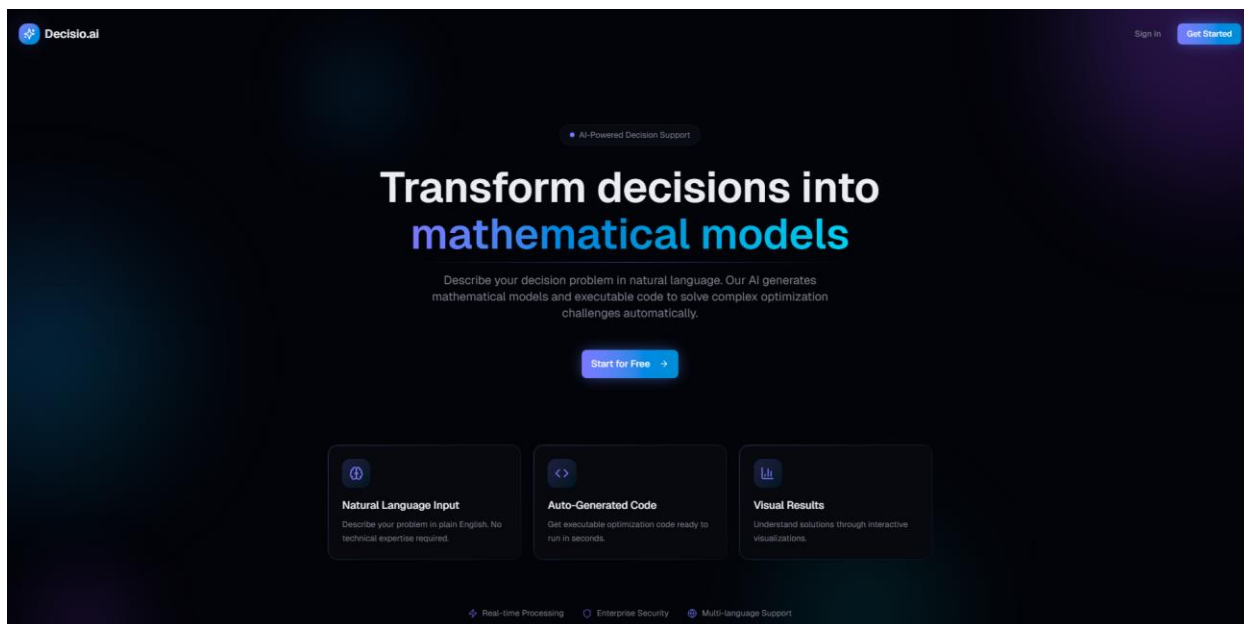
Linie kodu: 31626 łącznie (w tym 11771 “.ts”, 3499 “.java”, 1916 “.py”)

Liczba testów jednostkowych: 29 backend + 15 agents

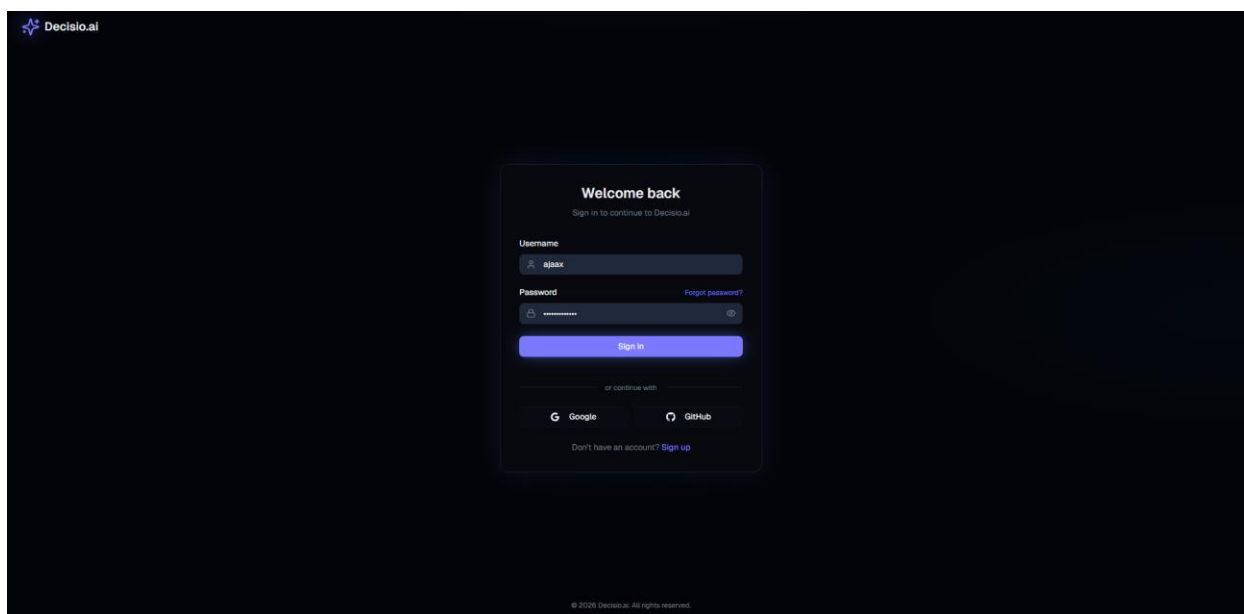
7 Projekt standardu interfejsu użytkownika

Warstwa prezentacyjna to aplikacja Frontend w technologii React.

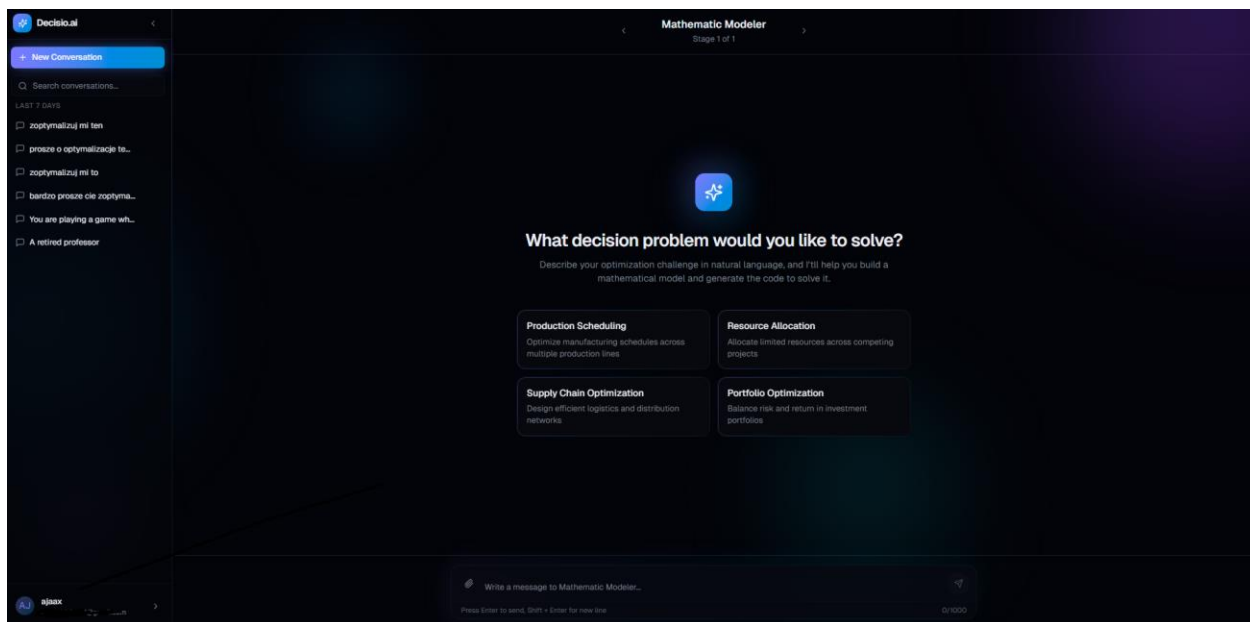
Główne widoki:



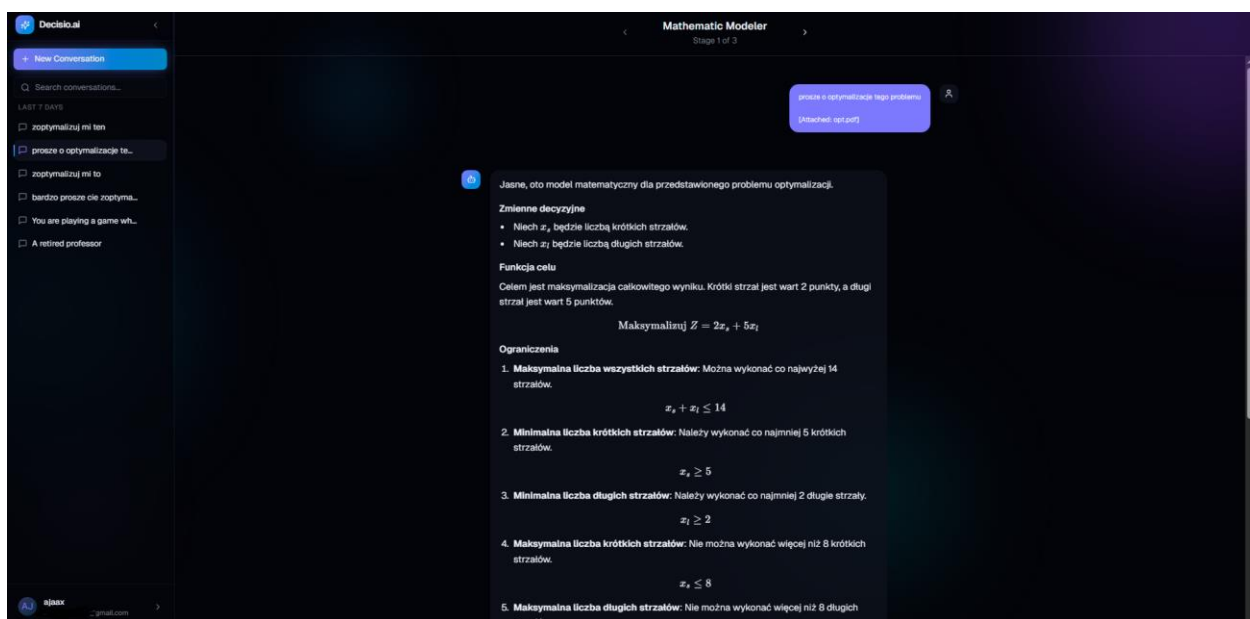
Strona główna



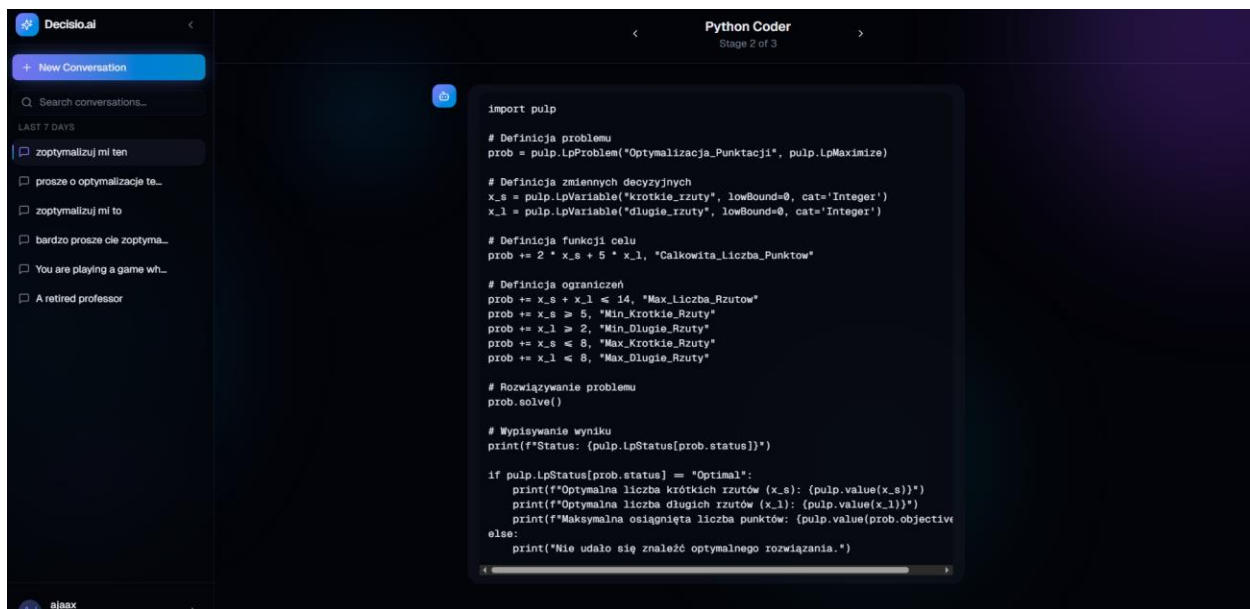
Strona logowanie



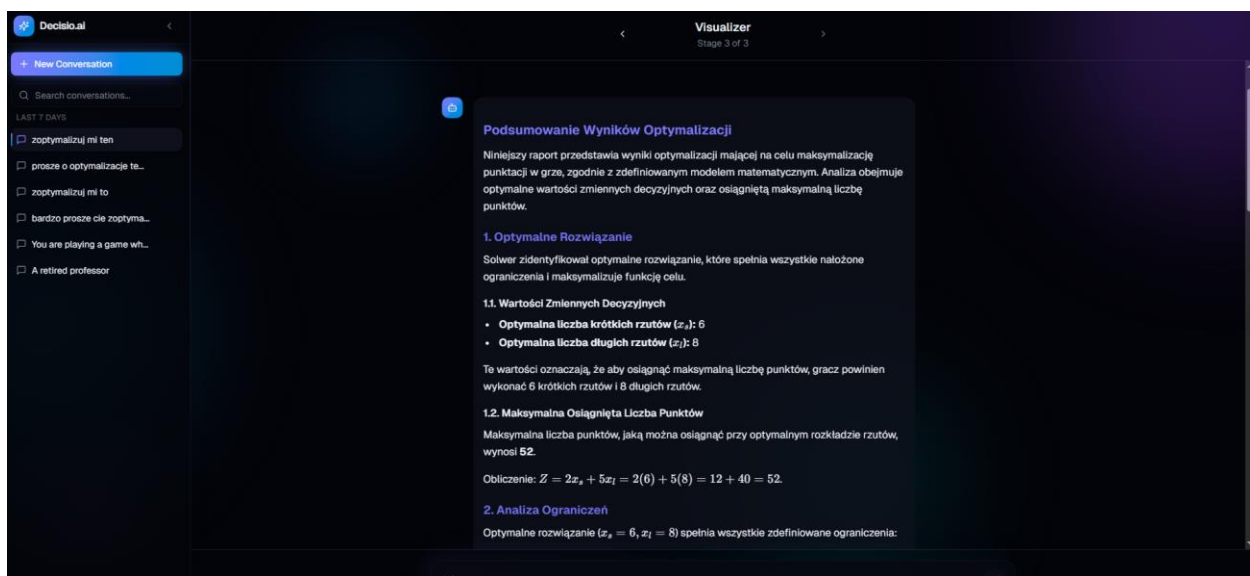
Strona z tworzeniem konwersacji



Widok chatu: modeler



Widok chatu: coder



Widok chatu: visualizer (1/2)



Widok chatu: vizualizer (2/2)

8 Specyfikacja testów

System realizuje strategię Defensive Programming na obu warstwach aplikacji, zapewniając stabilność i przejrzystość działania:

- **Warstwa Frontend (Client-Side Validation):** Aplikacja kliencka (React) wymusza poprawność danych jeszcze przed wysłaniem żądania do serwera. Wykorzystując schematy walidacyjne system blokuje interfejs w przypadku wykrycia nieprawidłowości.
- **Warstwa Backend (Server-Side Logging & Handling):** Serwis backendowy posiada scentralizowany mechanizm obsługi wyjątków (Global Exception Handler).

Rodzaje testów:

- Testy jednostkowe: Sprawdzają działanie danych metod / funkcji.
- Testy integracyjne: Sprawdzenie komunikacji między komponentami (poprawny przesył danych z frontendu do backendu a następnie do agentów)
- Testy akceptacyjne: Zgodnie ze scenariuszami użycia, wykonane przez zewnętrzne osoby – przejście pełnej ścieżki od zadania pytania biznesowego, przez akceptację modelu, aż po wynik.
- Testy systemowe (End-to-End):
 - Scenariusz pozytywny ("Happy Path"): Przeprowadzono manualne testy dla zróżnicowanych problemów optymalizacyjnych. Potwierdzono poprawne przejście przez fazy Modelowania, Kodowania i Wizualizacji wraz z generacją wyników.
 - Obsługa błędów ("Error Handling"): Zweryfikowano zachowanie systemu w przypadku awarii. Potwierdzono, że system prezentuje czytelny komunikat o błędzie oraz umożliwia użytkownikowi ponowienie operacji (mechanizm "Retry").

9 Wirtualizacja/konteneryzacja

System został w pełni skonteneryzowany, co zapewnia powtarzalność środowiska uruchomieniowego i łatwość wdrożenia. Wykorzystano **Docker** oraz **Docker Compose**, w tym serwisy:

- database
- Rabbitmq
- Cache
- Sandbox-service
- Agent-service
- Backend-service
- Client

10 Bezpieczeństwo

Wdrożono następujące mechanizmy bezpieczeństwa:

- **Izolacja kodu (Sandboxing):** Najważniejszy element bezpieczeństwa. Kod wygenerowany przez LLM jest uruchamiany w odrębnym kontenerze Docker (sandbox), który ma ograniczone zasoby (CPU, RAM), brak dostępu do sieci internetowej oraz brak dostępu do systemu plików hosta (poza dedykowanym wolumenem tymczasowym). Chroni to przed atakami typu Remote Code Execution.
- **Zarządzanie sekretami:** Klucze API (np. GOOGLE_API_KEY) oraz hasła do bazy danych nie są przechowywane w kodzie, lecz w pliku .env, który nie jest wersjonowany w repozytorium (mechanizm .gitignore).
- **Backend API:** Endpointy API są zabezpieczone, a dane wejściowe od użytkownika podlegają walidacji (sanity check) przed przetworzeniem.

11 Podręcznik użytkownika

1. Utwórz konto na stronie aplikacji
2. Potwierdź email (wiadomość na wskazanym adresie)
3. Rozpocznij konwersację
4. Zatwierdź wyniki lub zażądaj zmian na etapach: modeler, coder, visualizer.

12 Podręcznik administratora

Wymagania:

- Docker, docker compose
- Klucze API do Google Gemini oraz bazy danych

Instrukcja:

1. Sklonuj repozytorium i przejdź do katalogu projektu conversational-system/ .
2. Utwórz plik .env, uzupełnij klucze "dummies".
3. Uruchom komendę "docker-compose up --build".
4. System będzie dostępny pod adresem: http://localhost:3000 (frontend) oraz http://localhost:8000 (API).

13 Podsumowanie

Co udało się zrealizować:

- Kompletny, edytowalny na każdym etapie proces rozwiązywania problemów optymalizacji od generacji modelu matematycznego i kodu, do wizualizacji
- Modułarna architektura aplikacji przy użyciu konteneryzacji, co ułatwia ewentualną rozbudowę.
- Środowisko sandbox zapewniające bezpieczeństwo przy uruchamianiu kodu

Czego nie udało się zrealizować:

- Wybór dostawcy API do agentów: z powodu konieczności wykupienia innych kluczy API do AI (np chatGPT czy lepsza wersja Google Gemini) skupiono się na implementacji wykorzystującej stałego dostawcę - darmowy Google Gemini.

14 Bibliografia

1. DOCKER INC. *Docker Documentation* [online]. [dostęp: 2025-11-02]. Dostępny w: <https://docs.docker.com/>
2. GOOGLE. *Google One AI* [online]. [dostęp: 2025-01-13]. Dostępny w: <https://one.google.com/about/ai-premium>
3. LANGCHAIN AI. *LangChain Python Documentation* [online]. [dostęp: 2025-01-02]. Dostępny w: https://python.langchain.com/docs/get_started/introduction
4. THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 17 Documentation* [online]. [dostęp: 2025-01-08]. Dostępny w: <https://www.postgresql.org/docs/17/index.html>
5. VMWARE. *Spring Boot Documentation* [online]. [dostęp: 2025-12-07]. Dostępny w: <https://docs.spring.io/spring-boot/index.html>

Zatwierdzam dokumentację.	 Data i podpis Mentora
---------------------------	--------------------------------